

```
private class MyAsyncTask extends AsyncTask<String, Integer,
Integer> {
    @Override
    protected void onProgressUpdate(Integer... progress) {
        // [... Update progress bar, Notification, or other UI
element ...]
    }

    @Override
    protected void onPostExecute(Integer... result) {
        // [... Report results via UI update, Dialog, or
notification ...]
    }
    @Override
    protected Integer doInBackground(String... parameter) {
        int myProgress = 0;
        // [... Perform background processing task, update
myProgress ...]
        PublishProgress(myProgress)
        // [... Continue performing background processing task ...]

        // Return the value to be passed to onPostExecute
        return result;
    }
}
```

As shown in the code, your subclass should implement the following event handlers:

- **doInBackground** Takes a set of parameters of the type defined in your class implementation. This method will be executed on the background thread, so it must not attempt to interact with UI objects.
- **Place your long-running code here**, using the **publishProgress** method to allow **onProgressUpdate** to post progress updates to the UI.
- **When your background task is complete**, return the final result for the **onPostExecute** handler to report it to the UI.
- **onProgressUpdate** Override this handler to post interim updates to the UI thread. This handler receives the set of parameters passed to **publishProgress** from within **doInBackground**.